Rumbo: API REST de tareas

Unidad 2 · Programación del lado del servidor
Bianchi Navarro · Informe técnico · 31 de octubre de 2026

1. Objetivo y solución

Se desarrolló una API para gestionar tareas personales con CRUD, persistencia en SQLite, sesiones de usuario y notificaciones en tiempo real. Node.js 22.20.0 y Express 5 atienden la API y un tablero web que permite registrar cuentas, crear, editar, buscar, exportar y eliminar tareas. El dominio de tareas está contemplado en la consigna; se eligió SQLite al no disponer de otra base especificada.

2. Diseño HTTP y organización funcional

El flujo es **ruta HTTP > servicio > repositorio SQLite**. La capa de dominio contiene validaciones Zod y funciones puras: normalización de títulos, lectura de versiones y transformación de filas. Estas producen nuevos valores sin modificar la entrada. La E/S se concentra en repositorios, sesiones y eventos; los servicios coordinan las operaciones.

Recurso y método	Comportamiento y respuesta
POST /api/tasks	Crea una tarea: 201, Location y ETag.
GET /api/tasks y /:id	Lista paginada o lectura individual: 200.
PUT / PATCH /api/tasks/:id	Reemplazo completo o cambio parcial: 200 y nueva versión.
DELETE /api/tasks/:id	Eliminación con control de versión: 204, sin cuerpo.
GET /api/tasks/export	Descarga JSON de las tareas del usuario: 200.

Los errores mantienen **{error:{code,message,details,requestId}}**. Se utilizan 400 para sintaxis/ID, 401 para autenticación, 403 para CSRF, 404 para recursos inexistentes, 405 para métodos no admitidos y 422 para validación. El contrato completo y los ejemplos están en docs/API.md.

3. Persistencia, validación y conflictos

El esquema relaciona **users**, **tasks** y **sessions**; utiliza tablas STRICT, claves foráneas, restricciones CHECK e índices. Todas las consultas reciben parámetros. El título admite 3-120 caracteres, la descripción hasta 2000, y estado/prioridad pertenecen a enumeraciones. Se rechazan campos desconocidos. La restricción UNIQUE por propietario y título normalizado devuelve **409** ante duplicados.

Para editar o eliminar se exige **If-Match** con el ETag actual: **428** si falta y **412** si está obsoleto. El UPDATE/DELETE compara la versión en la misma sentencia SQL, evitando que dos solicitudes sobrescriban silenciosamente los cambios. Cada consulta restringe el acceso al propietario autenticado.

4. Sesiones, archivos y comunicación en tiempo real

El registro e inicio de sesión regeneran el identificador. Express-session conserva las sesiones en SQLite y envía una cookie **HttpOnly**, **SameSite=Strict**, con una hora de duración; en producción exige HTTPS mediante cookie Secure y secreto estable. Las contraseñas usan **scrypt** con sal aleatoria. Las escrituras exigen token CSRF y los intentos de autenticación se limitan por IP. Al salir se revoca la sesión y se cierran sus conexiones SSE.

GET /api/events abre un flujo Server-Sent Events. Después de confirmar cada escritura se emite **task.changed** con tipo created, updated o deleted, únicamente a las sesiones del propietario. Un heartbeat verifica las sesiones cada 15 segundos. Al reconectar, el tablero consulta nuevamente los datos porque los eventos no se almacenan. La exportación descarga un archivo JSON autorizado, sin contraseñas ni datos de sesión.

5. Pruebas y resultados

La suite de Node.js ejecutó **27 pruebas: 27 aprobadas, 0 fallos**. Utiliza HTTP real, SQLite aislado y conexiones SSE. Comprueba CRUD, validación, errores, duplicados, ediciones simultáneas, aislamiento por usuario, CSRF, expiración/logout, exportación y eventos. Otra prueba cierra y reabre la base para verificar tareas y sesiones con un secreto estable. Las transformaciones puras se verifican con una entrada inmutable.

Se incluye una colección Postman de **24 solicitudes**, una transcripción de **16 operaciones HTTP reales**, resultados de pruebas y capturas. En el navegador se verificaron registro, acceso, creación y recepción del cambio en una segunda pestaña sin recargar. GitHub Actions ejecuta la suite sobre Node.js 22.20.0 al publicar cambios.

6. Reproducción y alcance

Después de clonar el repositorio:

```
npm ci
npm test
npm run demo
```

Abrir **http://127.0.0.1:3000**; usuario **demo**, clave de ejemplo **RumboDemo2026!**. El modo demo escucha solo en localhost y crea datos de muestra. Para una cuenta propia: npm start. SQLite conserva los datos; las sesiones entre reinicios requieren SESSION_SECRET estable. La solución es de un solo proceso: un despliegue público necesita HTTPS y disco persistente; varias instancias requieren un bus de eventos y limitador compartidos. No se ha desplegado un servidor público.

7. Enlaces de entrega

Repositorio:
https://github.com/bianchi-navarro/tareas-api-aplicaciones-web-u2

Demostración local, guion y evidencias:
https://github.com/bianchi-navarro/tareas-api-aplicaciones-web-u2/blob/main/docs/DEMOSTRACION.md

Referencias

Express.js. (s. f.). *5.x API reference.* https://expressjs.com/en/5x/api/
Express.js. (s. f.). *Session middleware.* https://expressjs.com/en/resources/middleware/session/
Node.js. (s. f.). *SQLite: Node.js v22.20.0 documentation.*
https://nodejs.org/download/release/v22.20.0/docs/api/sqlite.html